System Architecture Overview

Purpose of This Page

This is the map. Every other page in this project documents one component or one decision in depth — this page exists to show how those pieces connect, so a new reader can understand the shape of the whole system before diving into any single part. If you're new to this project, start here.

What This System Is

An observability pipeline that captures LLM traffic, stores it semantically, and automatically detects sessions that drift from normal use into prompt-injection or jailbreak attempts. It is explicitly **not** a real-time blocking system — every component past the gateway's inline filter runs asynchronously, and a failure anywhere in this pipeline never affects the user-facing request/response path.

Component Map

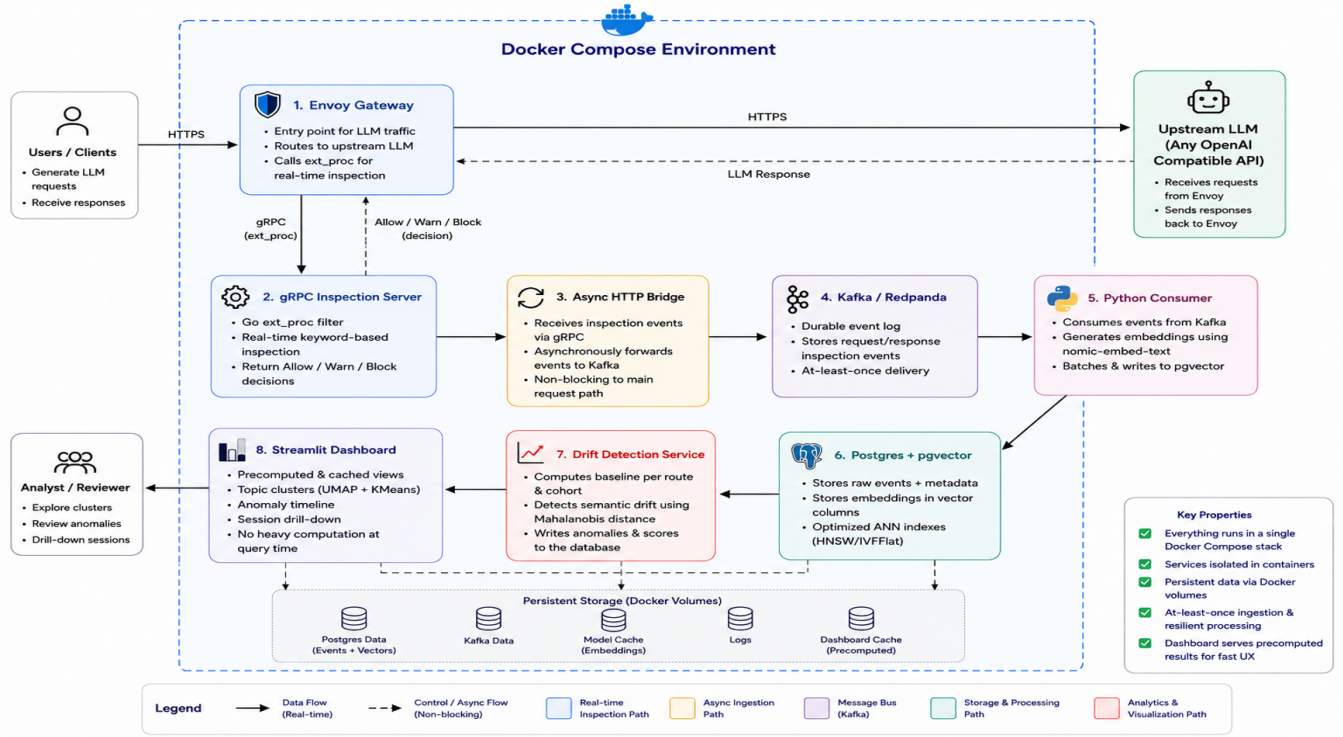
Component	Role	Owner
Envoy Gateway	Entry point for all LLM traffic	Teammate
`grpc-inspection-server` (Go, `ext_proc` filter)	Real-time, inline keyword-based blocking	Teammate
Async HTTP Bridge	Decouples capture from the inline request path	Mine
Kafka / Redpanda	Event bus, at-least-once delivery	Mine
Python Consumer	Embeds every prompt/completion, writes to storage	Mine
Postgres / pgvector	Semantic storage, HNSW-indexed	Mine
Drift Detector	Scheduled baseline + online scoring jobs	Mine
Streamlit Dashboard	Visualization: clusters, timeline, drill-down	Mine

Everything from the HTTP Bridge onward is deployed as a single Docker Compose stack.

Data Flow

1. A client request hits **Envoy Gateway**. The `ext_proc` filter inspects it inline and applies real-time keyword-based blocking — this is the only synchronous, request-path component in the whole system.
2. The captured payload (independent of whether it was blocked) is forwarded asynchronously via the **HTTP Bridge** to **Kafka/Redpanda**.
3. The **Python Consumer** reads events off the topic, batches them (32 at a time), embeds prompt and completion text separately via `nomic-embed-text`, and writes both the raw event and its embeddings to **Postgres/pgvector**.
4. The **Drift Detector** runs two independent, scheduled processes against this stored data:
    - A **baseline job** that periodically (re)computes what "normal" looks like per route, using a multi-cluster model (see the Multi-Cluster Baseline page).
    - An **online scoring job** that compares new sessions against the current baseline, using diagonal Mahalanobis distance (see the Mahalanobis Distance page), and separately checks new prompts against a permanent memory of confirmed past attacks (see the Persistent Memory page).
5. Flagged sessions are written to a `drift_events` table.

6. The **Streamlit Dashboard** reads directly from Postgres — embeddings, cluster assignments, and flagged events — to render three pages: Topic Clusters, Anomaly Timeline, and Session Drill-down.



Why Everything Past the Gateway Is Asynchronous

This is a deliberate reliability property, not an incidental design choice. If Kafka, the consumer, the detector, or the dashboard all went down simultaneously, a user talking to the LLM through the gateway would notice nothing — the inline `ext_proc` filter is the only component in the critical path. This fail-open design is why the system is described as observability, not protection: it trades the ability to block in real time for the ability to run a much more expensive, much more thorough semantic analysis without any latency cost to the user.

Reliability Properties Worth Knowing

Two infrastructure-level guarantees are load-bearing for this architecture to actually hold up under continuous operation, not just in a demo:

- **At-least-once Kafka delivery** means duplicate event delivery is an expected, handled case — the consumer's writes to Postgres are idempotent (unique constraint on Kafka topic/partition/offset).
- **Migration idempotency** (Postgres advisory lock + migration ledger) ensures schema setup is safe even if multiple consumer replicas start simultaneously — see the HNSW Indexing & Retrieval, and Migration Idempotency page for the full mechanism and how it was verified.